

Perancangan Smart Server System Berbasis Monitoring Suhu, Kelembapan, dan Keamanan Akses di CV Gundara Solusi Bersama Ungaran

Aizaas Vianansa Rochmawan¹

¹Program Studi S1 Sistem Komputer, Universitas Sains dan Teknologi Komputer, Semarang, Indonesia
Email: aizaas.vr@student.stekom.ac.id

Article Info	Tanggal: Received 3 March 2026, Revised 3 March 2026, Accepted 3 March 2026
Jenis Naskah	Artikel implementasi sistem berbasis IoT untuk monitoring dan keamanan rak server
Konteks Studi	CV Gundara Solusi Bersama, Ungaran

Abstract

Pengelolaan rak server menuntut kontrol termal dan keamanan akses fisik yang stabil, terukur, dan dapat di-audit. Kondisi studi menunjukkan kebutuhan integrasi antara pemantauan suhu-kelembapan, kendali pendinginan bertingkat, autentikasi akses, serta pencatatan log terpusat. Penelitian ini menyusun implementasi *Smart Server System* berbasis ESP32 dengan sensor SHT21, keypad 4x4, relay, kipas DC, solenoid lock, dashboard lokal, dan logging cloud melalui Google Apps Script–Google Sheets. Metode mencakup perancangan arsitektur perangkat keras-perangkat lunak, penetapan kontrak API lokal, implementasi manajemen antrian log, serta validasi berbasis skenario uji skrip. Hasil implementasi kode menunjukkan logika termal berjalan sesuai konfigurasi default (*warn* 27.0°C, *stage2* 28.0°C), lockout keamanan aktif setelah tiga kegagalan autentikasi, dan mekanisme *retry backoff* pada pengiriman log cloud. Eksekusi 12 skenario skrip uji pada 3 March 2026 menghasilkan 7 skenario sukses, 1 gagal koneksi, dan 4 timeout, yang mengindikasikan fungsi inti sistem berjalan namun reliabilitas jalur jaringan eksternal masih perlu penguatan. Kontribusi naskah ini adalah menyediakan draft jurnal yang lebih lengkap, berbasis artefak kode aktual, dan siap dilanjutkan ke pengujian lapangan terstruktur.

Keywords: akses fisik, ESP32, kelembapan, monitoring suhu, smart server

1 INTRODUCTION

Rak server merupakan komponen kritis infrastruktur teknologi informasi karena menampung perangkat yang beroperasi kontinyu pada beban dinamis. Standar operasional data center menekankan batas lingkungan ter-

mal tertentu agar perangkat tidak mengalami penurunan kinerja maupun kegagalan prematur [1, 2]. Selain dimensi termal, aspek keamanan fisik harus mampu menjaga prinsip *confidentiality*, *integrity*, dan *availability* [3].

Dalam konteks studi, perangkat server menangani data operasional proyek strategis sehingga dibutuhkan sistem monitoring yang tidak hanya menampilkan suhu-kelembapan, namun juga memberikan reaksi otomatis terhadap anomali. Draft Proposal Rev 2 mendokumentasikan bahwa pendekatan sebelumnya masih bergantung pada pendinginan ruangan, belum memiliki penguncian adaptif terhadap percobaan akses ilegal, dan belum menata jejak audit akses secara konsisten [4].

Penelitian ini mengadopsi pendekatan implementatif: memetakan kebutuhan operasional ke modul firmware ESP32, layanan jaringan lokal, dan backend logging cloud. Sistem dikembangkan menggunakan stack yang telah terdokumentasi di repositori proyek, termasuk PlatformIO, Arduino framework, ArduinoJson, serta integrasi Apps Script [5, 6, 7, 8].

Agar sejalan dengan format jurnal, naskah ini tidak berhenti pada deskripsi komponen, melainkan menyajikan model kontrol, kontrak API, matriks skenario pengujian, hasil eksekusi aktual, serta keterbatasan teknis yang masih harus dituntaskan.

Kontribusi kerja dirangkum sebagai berikut:

1. Menyusun model kendali termal bertingkat berbasis threshold dengan parameter yang dapat dikonfigurasi.
2. Mengimplementasikan keamanan akses berbasis PIN hash SHA-256 dengan lockout otomatis.
3. Menetapkan kontrak endpoint REST dan skema payload untuk *telemetry_logs* dan *access_logs*.
4. Menyajikan evaluasi berbasis skenario uji skrip yang dapat direplikasi.

2 THEORETICAL BASIS AND PROPOSED ARCHITECTURE

2.1 Model Kontrol Termal

Sistem menggunakan dua ambang suhu: T_{warn} dan T_{stage2} . Dengan suhu aktual $T(t)$ dan validitas sensor

$V(t)$, status peringatan didefinisikan sebagai:

$$W(t) = \begin{cases} 1, & \text{jika } V(t) = 1 \wedge T(t) > T_{warn} \\ 0, & \text{lainnya} \end{cases} \quad (1)$$

Aktivasi kipas tahap 2:

$$F_2(t) = \begin{cases} 1, & \text{jika } V(t) = 1 \wedge T(t) \geq T_{stage2} \\ 0, & \text{lainnya} \end{cases} \quad (2)$$

Aktivasi kipas tahap 1 mengikuti baseline sistem:

$$F_1(t) = B \vee W(t) \vee F_2(t) \quad (3)$$

dengan B adalah *fanBaselineOn*. Implementasi ini konsisten dengan logika pada firmware [6].

2.2 Model Keamanan Akses

Untuk setiap percobaan autentikasi ke- k , status akses didefinisikan:

$$A_k = \begin{cases} \text{GRANTED}, & \text{jika hash(PIN) = hash tersimpan} \\ \text{DENIED}, & \text{jika tidak cocok} \end{cases} \quad (4)$$

Jika jumlah kegagalan berurutan mencapai N_{max} , sistem masuk state lockout selama L detik:

$$\sum_{i=1}^m [A_i = \text{DENIED}] \geq N_{max} \Rightarrow \text{LOCKOUT}(L) \quad (5)$$

Nilai default implementasi adalah $N_{max} = 3$ dan $L = 120$ detik [6].

2.3 Arsitektur Usulan

Secara konseptual, arsitektur dibagi menjadi empat lapisan: sensor-input, kontrol-aktuator, layanan jaringan, dan logging cloud. Ringkasan komponen disajikan pada Tabel 1.

Table 1: Komponen utama dan peran fungsional

Komponen	Peran
ESP32	Pengendali utama, host API lokal, orkestrasi modul <i>sensor-access-network</i> .
SHT21	Akuisisi suhu dan kelembapan melalui I2C.
Keypad 4x4	Input PIN pengguna/admin untuk autentikasi.
Relay CH1-CH2	Kendali fan pendinginan tahap 1 dan tahap 2.
Relay CH3 + Solenoid	Kendali buka-kunci pintu berdasarkan hasil autentikasi.
LCD 20x4	Monitoring lokal (telemetri, status akses, lockout).
Google Apps Script	Endpoint logging cloud untuk <i>telemetry_logs</i> dan <i>access_logs</i> .

3 METHOD

3.1 Pendekatan Pengembangan

Pengembangan dilakukan secara iteratif dengan tahapan: analisis kebutuhan, implementasi modul, integrasi antar-modul, validasi skenario, dan dokumentasi hasil. Pendekatan ini dipilih agar perubahan konfigurasi dan perilaku sistem dapat diverifikasi bertahap pada level fungsi.

3.2 Sumber Data dan Instrumen Validasi

Sumber data penelitian dibagi menjadi tiga:

- Hasil implementasi kode:** analisis langsung file firmware dan konfigurasi [6].
- Hasil simulasi script test:** skenario pengiriman payload di folder *tests* [9].
- Placeholder uji lapangan:** ruang untuk metrik yang belum tersedia saat penyusunan naskah.

Instrumen evaluasi meliputi pemeriksaan konsistensi logika, kelengkapan field payload, stabilitas state keamanan, dan status respons layanan cloud.

3.3 Kontrak Endpoint REST

Endpoint lokal yang diimplementasikan ditunjukkan pada Tabel 2. Format ini disusun lebih rinci agar mudah direplikasi saat pengujian atau integrasi dashboard.

Table 2: Kontrak endpoint REST lokal

Method	Route	Fungsi
GET	/api/state	Mengembalikan telemetri, status fan/alarm, lockout, RSSI, dan antrean log.
GET/POST	/api/config/thermal	Membaca/mengubah threshold termal, baseline fan, interval sensor/cloud.
GET/POST	/api/config/security	Membaca/mengubah batas gagal, durasi lockout, unlock solenoid, device ID.
GET/POST	/api/users	Daftar pengguna dan upsert user berbasis PIN hash.
DELETE	/api/users/{id}	Menghapus user berdasarkan <i>userId</i> .
POST	/api/send	Memaksa flush antrean akses+telemetri ke cloud.
GET	/api/wifi/scan	Pindai jaringan WiFi sekitar dan status saved/open.
POST	/api/wifi/connect	Simpan kredensial dan lakukan upaya koneksi STA.

3.4 Desain Skenario Uji

Matriks skenario mengikuti file *test_*.py* pada repositori. Daftar skenario disajikan pada Tabel 3.

3.5 Kriteria Evaluasi

Kriteria evaluasi dibagi menjadi:

- Konsistensi logika:** perilaku modul sesuai aturan implementasi di source code.
- Kelengkapan data:** field payload sesuai kontrak *telemetry_logs/access_logs*.

Table 3: Matriks skenario uji berbasis script

File Skenario	Tujuan Uji
test_telemetry_normal.py	Validasi payload kondisi normal.
test_telemetry_warning.py	Validasi payload saat suhu melewati threshold peringatan.
test_telemetry_alarm.py	Validasi payload alarm suhu kritis.
test_telemetry_fans_off.py	Validasi kondisi fan nonaktif.
test_telemetry_high_humidity.py	Validasi payload kelembapan tinggi.
test_telemetry_rssi_good.py	Validasi pelaporan RSSI kuat.
test_telemetry_rssi_bad.py	Validasi pelaporan RSSI lemah.
test_telemetry_door_unlock.py	Validasi status pintu unlock pada payload telemetry.
test_access_granted.py	Validasi log akses diterima.
test_access_denied.py	Validasi log akses ditolak.
test_access_lockout.py	Validasi log lockout saat gagal berulang.
test_access_unknown_user.py	Validasi log user tidak dikenal.

3. **Stabilitas eksekusi:** skenario skrip mampu mengembalikan respons JSON `{"ok":true}`.
4. **Transparansi hasil:** jika ada timeout atau kegagalan koneksi, dicatat sebagai temuan, bukan disembunyikan.

4 RESULTS AND DISCUSSION

4.1 Hasil Implementasi Kode

Audit terhadap modul inti memperlihatkan keterhubungan antarlayer yang jelas.

(1) **Layer termal dan aktuasi.** Firmware memproses validitas sensor, menghitung state peringatan, lalu mengendalikan dua fan sesuai aturan threshold. Pendekatan ini menjaga transisi dari kondisi normal ke kondisi kritis secara deterministik.

(2) **Layer keamanan.** Proses autentikasi menggunakan hash SHA-256, pencatatan event akses, dan lock-out otomatis setelah batas percobaan gagal. State keamanan ini disinkronkan ke UI lokal serta endpoint status.

(3) **Layer jaringan dan cloud logging.** Sistem memiliki queue terpisah untuk telemetry dan akses, prioritas kirim log akses, serta *exponential backoff*. Strategi ini penting untuk menahan kehilangan log saat koneksi tidak stabil.

4.2 Hasil Eksekusi Aktual Skenario Script Test

Eksekusi aktual dilakukan pada **Selasa, 3 March 2026** dengan wrapper timeout 25 detik per skrip. Ringkasan hasil ditunjukkan pada Tabel 4.

Table 4: Ringkasan hasil eksekusi 12 skenario

Status	Jumlah
PASS (respons <code>{"ok":true}</code>)	7
FAIL (error koneksi)	1
TIMEOUT	4

Detail hasil per skenario disajikan pada Tabel 5.

Table 5: Detail status per file skenario

File	Status	Catatan
test_access_denied.py	PASS	JSON ok:true.
test_access_granted.py	PASS	JSON ok:true.
test_access_lockout.py	PASS	JSON ok:true.
test_access_unknown_user.py	TIMEOUT	Tidak selesai dalam 25 detik.
test_telemetry_alarm.py	PASS	JSON ok:true.
test_telemetry_door_unlock.py	FAIL	Connection reset by peer.
test_telemetry_fans_off.py	PASS	JSON ok:true.
test_telemetry_high_humidity.py	TIMEOUT	Tidak selesai dalam 25 detik.
test_telemetry_normal.py	PASS	JSON ok:true.
test_telemetry_rssi_bad.py	TIMEOUT	Tidak selesai dalam 25 detik.
test_telemetry_rssi_good.py	TIMEOUT	Tidak selesai dalam 25 detik.
test_telemetry_warning.py	PASS	JSON ok:true.

Hasil ini menunjukkan fungsi inti payload logger sudah berjalan, tetapi ketergantungan pada endpoint eksternal membuat reliabilitas eksekusi batch sensitif terhadap kondisi jaringan. Ini konsisten dengan desain sistem yang memang mengandalkan jalur internet untuk sinkronisasi cloud.

4.3 Pembahasan Teknis dan Implikasi

Dari sisi rekayasa sistem, terdapat tiga implikasi penting:

1. **Keandalan lokal lebih tinggi dari cloud path.** Modul kontrol termal-keamanan tetap deterministik karena berjalan di perangkat, sedangkan keberhasilan logging dipengaruhi availability jaringan.
2. **Queue dan retry backoff sudah tepat.** Mekanisme ini relevan untuk mencegah kehilangan log saat gangguan sementara, walaupun belum menggantikan kebutuhan observabilitas jaringan yang lebih dalam.
3. **Kesiapan audit sudah baik namun belum final.** Struktur field access log sudah cukup untuk jejak audit, tetapi kualitas data waktu nyata tetap bergantung pada stabilitas koneksi internet.

4.4 Keterbatasan dan Placeholder Uji Lapangan

Keterbatasan yang wajib dicatat pada tahap ini:

1. status pintu masih diturunkan dari event solenoid, belum dari sensor reed switch fisik,
2. HTTPS client masih memakai `setInsecure`,
3. metrik kuantitatif lapangan jangka panjang belum tersedia.

Rencana metrik lapangan disajikan pada Tabel 6.

Table 6: Placeholder metrik uji lapangan lanjutan

Metrik	Status	Keterangan
Waktu respons unlock	Placeholder	Dari submit PIN valid sampai relay solenoid aktif.
Stabilisasi suhu rak	Placeholder	Tren suhu sebelum-sesudah fan bertingkat aktif.
Reliabilitas pengiriman log 24 jam	Placeholder	Rasio sukses kirim terhadap total event.
Dampak kualitas RSSI pada latensi kirim	Placeholder	Pengukuran pada beberapa level RSSI terkontrol.

5 CONCLUSION

Naskah ini merevisi draft jurnal ke format yang lebih lengkap dan lebih dekat dengan pedoman template, dengan basis fakta dari source code, konfigurasi runtime, serta hasil eksekusi skenario uji yang terdokumentasi. Secara teknis, sistem telah menunjukkan integrasi fungsional antara monitoring termal, keamanan akses, API konfigurasi, dan logging cloud.

Eksekusi aktual pada 12 skenario skrip menunjukkan mayoritas skenario berhasil, namun masih ditemukan kegagalan/timeout pada sebagian skenario akibat jalur koneksi eksternal. Temuan ini mempertegas bahwa layer kontrol lokal sudah siap operasional, sementara layer cloud masih perlu penguatan reliabilitas end-to-end.

Pekerjaan lanjutan direkomendasikan pada: (1) integrasi sensor status pintu fisik, (2) penguatan TLS verification tanpa `setInsecure`, (3) kampanye uji lapangan terstruktur dengan metrik kuantitatif yang konsisten.

Acknowledgements

Terima kasih kepada pembimbing akademik, tim operasional, dan pihak CV Gundara Solusi Bersama yang menyediakan konteks implementasi serta kebutuhan sistem.

REFERENCES

- [1] ASHRAE, *Thermal Guidelines for Data Processing Environments*, 5th ed. American Society of Heating, Refrigerating and Air-Conditioning Engineers, 2021.
- [2] Telecommunications Industry Association, "Ansi/tia-942 telecommunications infrastructure standard for data centers," Standar industri, 2017.

- [3] W. Stallings, *Effective Cybersecurity: A Guide to Using Best Practices and Standards*. Addison-Wesley Professional, 2017.
- [4] A. V. Rochmawan, "Draft proposal rev 2: Perancangan smart server system berbasis monitoring suhu, kelembapan, dan keamanan akses di cv gundara solusi bersama ungaran," Dokumen internal (DOCX), 2026.
- [5] Smart Server Project Team, "Smart server system (esp32) - readme," Dokumen proyek internal, 2026, berkas: README.md pada repositori server.
- [6] —, "Firmware source code smart server esp32," Dokumen proyek internal, 2026, berkas: src/App.cpp, src/AccessController.cpp, src/NetworkServices.cpp, src/WiFiHandler.cpp, src/GoogleSheetsClient.cpp, src/Config.cpp.
- [7] PlatformIO, "Platformio documentation," <https://docs.platformio.org/>, 2026.
- [8] B. Blanchon, "Arduinjson documentation," <https://arduinjson.org/>, 2026.
- [9] Smart Server Project Team, "Simulation test scripts for telemetry and access logging," Dokumen proyek internal, 2026, berkas: tests/test_*.py dan tests/run_all.py.